Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 7

по курсу «Методы оптимизации»

«Метод Крэчти и Лева»

Студент группы ИУ9-82Б Шемякин В.А.

Преподаватель Посевин Д. П.

Москва 2026

1 Задание

Необходимо реализовать метод Крэчти и Лева.

2 Результаты

Исходный код программы представлен в листинге 1.

```
using LinearAlgebra
using Plots
using Colors
Plots.default(show=false)

schwefel(x) = 418.9829 * length(x) - sum(xi -> xi * sin(sqrt(abs(xi))), x)

function grad_schwefel(x)
    g = similar(x)
    for i in eachindex(x)
        xi = x[i]
        s = sqrt(abs(xi)) + 1e-20
        g[i] = xi >= 0 ? -sin(s) - xi * cos(s) / (2 * s) : -sin(s) + xi * cos(s) / (2 * s)
    end
    return g
end

function minimize_1d(f, a, b, tol=1e-8)
    for _ in 1:100
        if b - a < tol
            return (a + b) / 2
        end
        c = (2a + b) / 3
        d = (a + 2b) / 3
        if f(c) < f(d)
            b = d
        else
            a = c
        end
    end
    return (a + b) / 2
end

```

```
function numgrad(f, x, h=1e-7)
    n = length(x)
    g = zeros(n)
    for i in 1:n
        xp = copy(x)
        xm = copy(x)
        xp[i] += h
        xm[i] -= h
        g[i] = (f(xp) - f(xm)) / (2h)
    end
    return g
end

function line_search_bounds(x, d, ranges)
    n = length(x)
    lo, hi = 0.0, 1.0
    for i in 1:n
        if abs(d[i]) < 1e-12
            continue
        end
        r = ranges[i]
        t1 = (r[1] - x[i]) / d[i]
        t2 = (r[2] - x[i]) / d[i]
        t_lo, t_hi = min(t1, t2), max(t1, t2)
        lo = max(lo, t_lo)
        hi = min(hi, t_hi)
    end
    lo = max(lo, 0.0)
    return lo, hi
end

function fletcher_reeves(g, grad_g, x0, ranges, tol=1e-8, max_iter
    =200)
    n = length(x0)
    x = copy(x0)
    gx = grad_g(x)
    d = -gx
    g_prev_norm_sq = dot(gx, gx)
    for _ in 1:max_iter
        if norm(gx) < tol
            break
        end
        lo, hi = line_search_bounds(x, d, ranges)
        if hi <= lo + 1e-10
            break
```

```
        end
        phi(alpha) = g(x + alpha * d)
        alpha = minimize_1d(phi, lo, hi)
        x = x + alpha * d
        for i in 1:n
            x[i] = clamp(x[i], ranges[i][1], ranges[i][2])
        end
        gx_new = grad_g(x)
        beta = dot(gx_new, gx_new) / (g_prev_norm_sq + 1e-20)
        g_prev_norm_sq = dot(gx_new, gx_new)
        d = -gx_new + beta * d
        gx = gx_new
    end
    return x
end

function fletcher_reeves_minimize(g, ranges, tol=1e-8)
    n = length(ranges)
    x0 = [(r[1] + r[2]) / 2 for r in ranges]
    grad_g = x -> numgrad(g, x)
    return fletcher_reeves(g, grad_g, x0, ranges, tol)
end

function krechti_leva(f, grad_f, x0, eps, n_lambdas; max_iter
    =10000)
    dim = length(x0)
    m = n_lambdas - 1
    x = copy(x0)
    delta_history = [zeros(dim) for _ in 1:max(1, m)]
    traj = [copy(x)]
    iter_count = 0

    for k in 1:max_iter
        iter_count = k
        if k > 0 && k % 10 == 0
            delta_history = [zeros(dim) for _ in 1:max(1, m)]
        end

        g = grad_f(x)
        if norm(g) < eps
            break
        end

        obj(lambda) = begin
            s = -lambda[1] * g
```

```
            for i in 1:m
                s += lambda[i+1] * delta_history[i]
            end
            f(x + s)
        end

        r = max(0.1, min(1.0, norm(g) > 1e-10 ? 1.0 / norm(g) : 1.0))
        ranges = [(0.0, r * 2.0)]
        for _ in 2:n_lambdas
            push!(ranges, (-r * 0.5, r * 0.5))
        end

        lambda_opt = fletcher_reeves_minimize(obj, ranges)

        step = -lambda_opt[1] * g
        for i in 1:m
            step += lambda_opt[i+1] * delta_history[i]
        end
        x_new = x + step

        for i in m:-1:2
            delta_history[i] = delta_history[i-1]
        end
        delta_history[1] = x_new - x

        x = x_new
        push!(traj, copy(x))

        if norm(delta_history[1]) < eps
            break
        end
    end

    return x, f(x), traj, iter_count
end

function draw_trajectory_2d(f, traj, title_str, filename; lw=4)
    xs = [p[1] for p in traj]
    ys = [p[2] for p in traj]
    fv = [f(p) for p in traj]
    rng = max(maximum(fv) - minimum(fv), 1e-10)
    span_x = maximum(xs) - minimum(xs)
    span_y = maximum(ys) - minimum(ys)
    pad = 0.2 * max(span_x, span_y, 0.5)
```

```
    xlim = (minimum(xs) - pad, maximum(xs) + pad)
    ylim = (minimum(ys) - pad, maximum(ys) + pad)
    xg = range(xlim[1], xlim[2], length=80)
    yg = range(ylim[1], ylim[2], length=80)
    plt = contour(xg, yg, (x, y) -> f([x, y]), levels=20, color=cgrad(:viridis, alpha=0.6), colorbar=true, legend=false, xlabel="x_1", ylabel="x_2", title=title_str, xlims=xlim, ylims=ylim)
    plot!(plt, xs, ys, color=:white, lw=lw+2, label="")
    for i in 1:length(traj)-1
        t = (fv[i] - minimum(fv)) / rng
        col = RGB(0.1 + 0.8*(1-t), 0.2 + 0.6*(1-t), 0.9 - 0.7*t)
        plot!(plt, xs[i:i+1], ys[i:i+1], color=col, lw=lw, label="")
    end
    scatter!(plt, [xs[1]], [ys[1]], color=:lime, ms=12, marker=:star5, label="            ")
    scatter!(plt, [xs[end]], [ys[end]], color=:red, ms=12, marker=:diamond, label="            ")
    savefig(plt, filename)
end

eps = 1e-6
n_lambdas_list = [2, 3, 5, 10, 100]
x0 = [320.0, 350.0]

for n in n_lambdas_list
    x, fv, traj, it = krechti_leva(schwefel, grad_schwefel, x0, eps, n)
    println("n = ", n, ":")
    println("  x* = ", round.(x, digits=4))
    println("  f(x*) = ", round(fv, digits=10))
    println("                    = ", it)
    title_str = "                2D      n=$n    ,                    =$it"
    filename = joinpath(@__DIR__, "schwefel_n$(n)_trajectory.png")
    draw_trajectory_2d(schwefel, traj, title_str, filename)
end
```

Результат представлен на рисунках 1– 3.

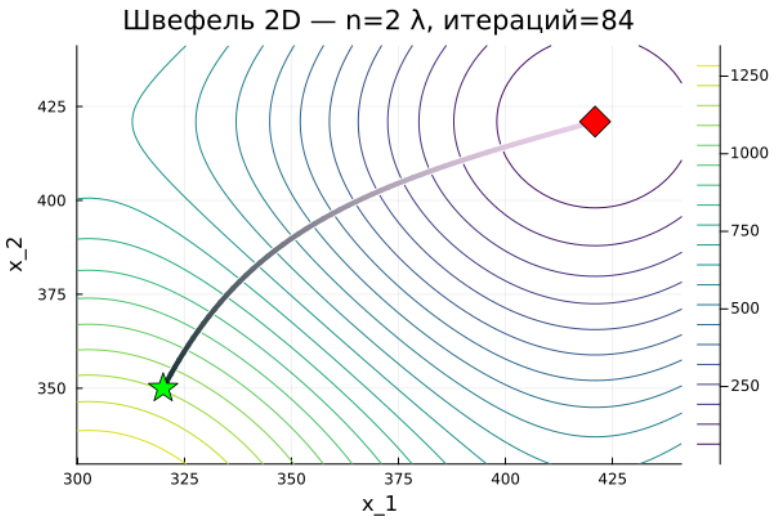

Рис. 1 — Результат

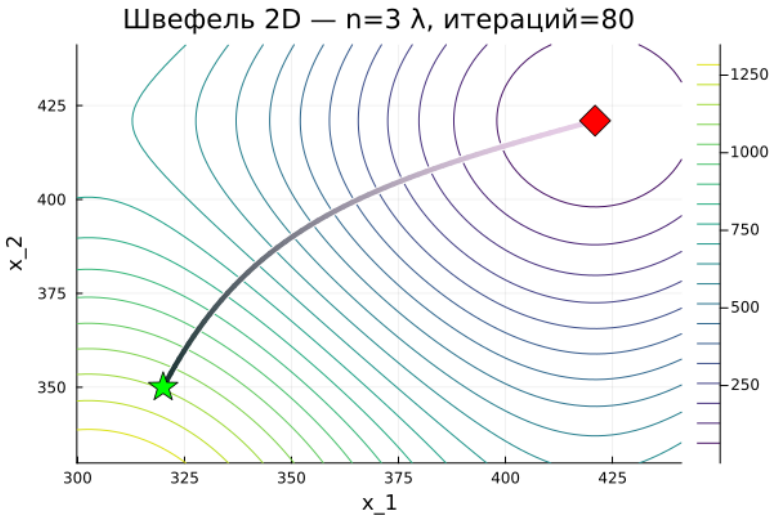

Рис. 2 — Результат

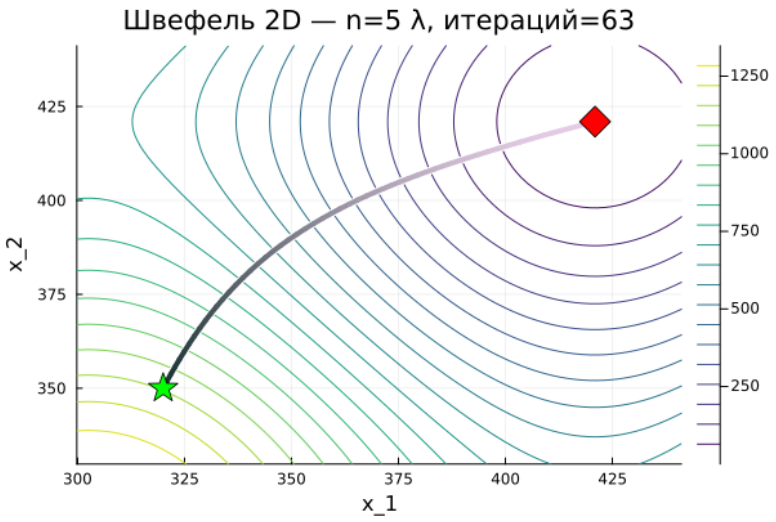

Рис. 3 — Результат